

Musical Lick Modifier: CPSC 490 Senior Thesis

Max Su
max.su@yale.edu

Advisor: Scott Petersen
scott.petersen@yale.edu

*A Senior Thesis as a partial fulfillment of requirements
for the Bachelor of Science in Computer Science*

Department of Computer Science
Yale University
May 1, 2025

Acknowledgements

I would like to acknowledge Professor Scott Petersen for his support and guidance through this project. Special thanks to the Yale Department of Computer Science for providing a fantastic environment for academic growth. I'm grateful for the opportunities and resources that have enriched my time here!

Contents

1	Introduction	5
2	Background	6
2.1	Improvisation in Bebop	6
2.2	Computer Music Models	6
2.3	Symbolic vs Audio Music Generation	7
2.4	Markov Chains	8
3	Methodology	9
3.1	Design	9
3.2	Parser	10
3.3	Creator	11
3.4	Embellisher	12
4	Results	13
4.1	Current User Experience	13
4.2	Examples	13
4.3	Discussion	14
5	Related Work	15
6	Conclusions	16
7	Future Work	17

Musical Lick Modifier

Max Su

Abstract

This project proposes a rule-based system for modifying improvisational licks through MIDI. Targeted toward aspiring jazz musicians, the tool enables users to explore melodic regeneration and embellishment techniques without relying on large datasets or more self-learning models. By using Markov chains, the program can analyze both pitch class and rhythmic transitions within a given lick and will generate new variations that preserve its rhythmic and harmonic structure. The structure of the system is in three parts: the parser builds bar-specific transition matrices, while the creator samples pitches and rhythms from these matrices. Finally, the embellisher further refines the output by inserting idiomatic jazz gestures such as triplet turn motions.

This project also emphasizes ease of use, allowing musicians access to a fast tool for quick inspiration. The goal is to require as little input from the user as possible — no large training sets or data preparation — just a simple one-track MIDI file and chord progression. Users input a MIDI file and corresponding chord progression, and the program quickly outputs a modified lick, instantly opening the music in MuseScore. The model is designed to be educational rather than generative at scale — it is most effective when aiding musicians in creative ruts, providing new perspectives on how melodic lines can be constructed against a harmonic structure. By prioritizing transparency and stylistic adherence to bebop conventions, this project serves as a bridge between computational modeling and jazz pedagogy. Ultimately, it demonstrates how symbolic rule-based approaches can foster growth and enhance improvisational fluency in improvisational musicians.

1 Introduction

The computational generation and analysis of music has been a topic of discussion for a long period of time and is a constantly evolving landscape as ideas in mathematics and computer science are applied in novel ways. In this paper, we present a novel approach to short-form symbolic music generation that leverages Markov chains to craft a new melodic sequence while preserving the underlying melodic and rhythmic structure of the input.

Improvisation is at the heart of jazz music, yet many aspiring jazz musicians find themselves encountering creative barriers when developing their improvisational skills. The key to overcoming these barriers often lies in mastering the use of "licks" — short, practiced phrases used repeatedly during improvisations. Licks form the foundational vocabulary for jazz musicians, serving as reliable motifs or musical ideas that can be seamlessly integrated into solos. Initially, musicians learn and internalize simple licks, gradually expanding their repertoire with increasingly complex phrases. The other aspect of soloing in jazz is tying together licks and filling in other spaces.

This project introduces an innovative tool aimed at aspiring jazz musicians seeking to enrich their improvisational vocabulary. The project utilizes Markov chains to analyze and regenerate jazz licks, preserving their core rhythmic and melodic characteristics while introducing new music to the user by employing a custom-designed parser and generator developed in Python.

The goal is to provide musicians with an accessible, transparent, and practical tool for breaking through creative ruts, exploring new musical ideas, and gaining a deeper understanding of improvisational techniques. Ultimately, by blending mathematics and computer science with traditional jazz pedagogy, this work highlights how technology can enhance artistic growth and creativity in aspiring jazz musicians.

2 Background

2.1 Improvisation in Bebop

While different genres of music abide by their own specific set of conventions, jazz is unique in that there is a fusion of standard conventions and the unruly. While much of jazz is played within the chord changes, musicians are not afraid to push the boundaries during their experimentation.

Bebop is a style of jazz that emerged in the early 1940s as a departure from the more dance-oriented swing era. It is characterized by fast tempos, intricate melodies, and complex harmonic progressions. Bebop emphasizes individual virtuosity and spontaneous improvisation within small group settings, requiring musicians to navigate rapid chord changes and extended harmonies. These improvisations are often based on intricately stringing together licks that fit into these complex chord progressions.

However, one of the most important boundaries that Bebop pushed is its harmonic language: it is deeply rooted in the careful outlining of chord tones, which is considered the cornerstone of the style. Bebop musicians target the essential notes—typically the root, third, and seventh of each chord—to create solos that remain closely tied to the underlying harmony. This approach not only reinforces the chord changes but also provides a structural anchor for the fast, intricate improvisations characteristic of bebop. As time continued and the more Bebop musicians experimented, the more "out" they became: meaning playing and emphasizing notes outside the chord. While bebop lines feature chromatic passing tones and "out" notes, the emphasis on chord tones ensures that the improvisation is both coherent and connected to the tune's harmonic framework.

Because of its strong ties harmonic progressions and emphasis on outlining chord tones, Bebop is an advantageous genre to learn to play sooner rather than later in a jazz musicians journey. It requires a deep understanding of the chords, flexibility, rhythmic integrity, and technical ability. The lick modifier assists musicians in their journey to understand the chord progression at a deeper level: finding out which notes sound good and which don't while having a program continually output ideas for improvement.

2.2 Computer Music Models

There are two primary approaches to computer generated music: self-learning systems and rule-based systems.

Self-learning systems rely on machine learning methods to discover patterns in music from a specific set of training data. With the recent popularity in deep learning and AI technology, there has been a rise in the application of self-learning musical analysis and generation systems. However, these systems allow for minimal human intervention as the creativity is driven solely by the data. One of the major disadvantages of a self-learning system is that when it fails, it breaks spectacularly, and due to the aforementioned lack of human intervention, there is little opportunity to correct it other than modifying the training data and run the model again.

In rule-based systems, the coder manually defines musical rules and implements their own algorithms. A rule-based system offers a more transparent, controllable, and interpretable approach to music generation. Unlike a self-learning system, which often operates as a "black box," rule-based systems allow for clearly-defined rules that will allow users to understand how their input will be transformed. Following a rule-based approach ensures that the modifications remain musically coherent while still providing unique licks that can spur the user's creativity.

2.3 Symbolic vs Audio Music Generation

Organizing musical data through symbolic music generation rather than audio-based generation has several key advantages in the context of rule-based systems.

Symbolic music, such as MIDI, represents music as discrete note events with clearly defined parameters: pitch, duration, velocity, and timing. This level of abstraction allows for precise manipulation and analysis of musical structure, making it ideal for tasks like generating licks that align with specific chord progressions, identifying chord tones, and implementing rule-based algorithms. It also allows for proper quantization of rhythms before inputting into the model, which allows it to return music in staff notation.

In contrast, audio-based generation requires learning or manipulating raw waveforms, which is computationally more intensive and far less interpretable. Moreover, extracting meaningful high-level musical features (like pitch classes, chord tones, and rhythm) from audio typically requires additional layers of complex signal processing or deep learning models, as well as precision from the user to play precisely. This introduces unnecessary complexity and noise into the system.

By using MIDI, this project remains grounded and allows for tight control over note-level decisions. This enables the system to generate jazz phrases that obey rules without relying on opaque machine learning models or black-box audio generation techniques. Ultimately, symbolic generation provides the ideal foundation for transparent, modular, and musically informed algorithmic composition.

2.4 Markov Chains

Markov chains are powerful mathematical models that can boil down a sequence of events in which the probability of each event solely depends on the previous state. All Markov chains operate under the Markov property: the future state depends exclusively on the current state instead of all the states before it. Mathematically, this property is noted as $P(X = x|x_1, x_2, \dots, x_n) = P(X = x|x_n)$, where $P(X = x)$ is the probability of the next state being x .

In the context of musical improvisation, Markov chains analyze sequences of notes or rhythms and quantify the likelihood of transitioning from one pitch or note length to another. These probabilities can be represented as transition matrices, which can then be utilized to generate new sequences that reflect the original data in a quantitative manner. The simplicity, computational efficiency, and clear interpretability of Markov chains make them particularly suitable for musical applications, allowing musicians to generate new ideas while preserving stylistic coherence.

3 Methodology

This research aimed to algorithmically generate improvisational licks from minimal music input using computational modeling. The most fundamental part of the project is that only the provided music itself is used as input data without having external training sets. Initially, we planned to use more complicated models such as a Hidden Markov Model or Grammars with compression algorithms to deconstruct the old lick and regenerate a new one. However, these approaches quickly demonstrated limitations due to the low amount of input data, leading to ineffective outcomes.

Consequently, it was deemed necessary to move to a simpler probabilistic model to fit the algorithmic and mathematical approach of the project. A first-order Markov Chain that simultaneously samples pitch class and rhythm. I wanted to use features of the new musicaiz Python package, so I was able to decompose the lick to its pitch class and note length transition matrices, build bar-specific probabilistic matrices out of them, weighting over both the entire lick and per chord change, then sample from them simultaneously to produce melodies.

The project was implemented in Python version 3.8.5 on a desktop platform, with visualizations and score editing conducted using MuseScore version 4.5.1. Essential dependencies included musicaiz for MIDI data handling, mido for editing MIDI files, and music21 for music visualization and analysis. Additional supporting packages included numpy for numerical computations, typing for type annotations, random and sys for program control, tkinter for user interface elements, and itertools for combinatorial and mathematical operations.

3.1 Design

The design of the project is simple and forward. The program takes in a MIDI file, runs it through the Parser class to build transition matrices for pitch class and note length, then samples the parser's joint Markov Chain in the Creator class. This output is then run through the embellisher before returning the modified MIDI file into the outputs folder as well as opening a new MuseScore file for visualization. In illustrating the design, we will walk through the process for a simply ii-V-I lick in the key of C major. This is a three-bar phrase that moves from the Dm7 minor seventh chord to a G7 dominant seventh chord to a Cmaj7 major seventh chord.

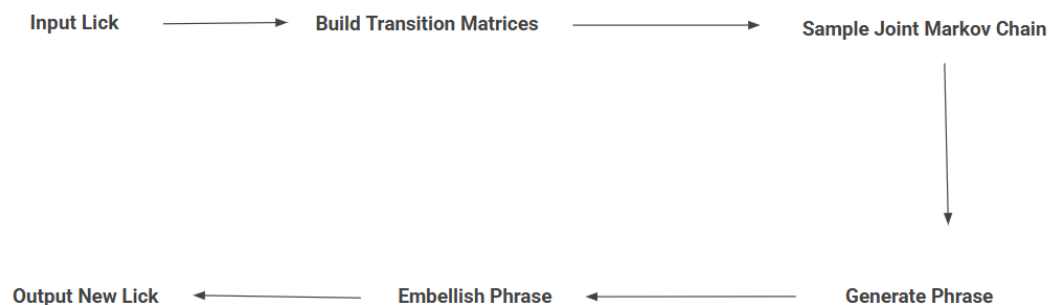


Figure 3.1: Flow chart for Entire Process

3.2 Parser

The parser is responsible for gathering information from the original lick, sifting for the most important information, and organizing it for the Creator.

For building the pitch class probabilistic transition matrix, the Parser utilizes Musicaiz's `pitch_class_transition_matrix` method, which tallies the frequency of transitions between different pitch classes. The rows are then normalized to sum to 1. Our novel approach to creating these transition matrices involves having a unique transition matrix for each bar that is reweighted by the entries in the transition matrix for the whole lick. Afterward, it identifies the notes in the scale (e.g. Dm7 = D E F G A Bb C) and sets all other columns to 0 so that only notes in the scale can be sampled.

For building the note-length probabilistic transition matrix, the parser utilizes Musicaiz's `note_length_matrix` and normalizes it. Currently, the model can only generate music with note lengths that appear in the piece.



Figure 3.2: This is the original lick in the MuseScore GUI

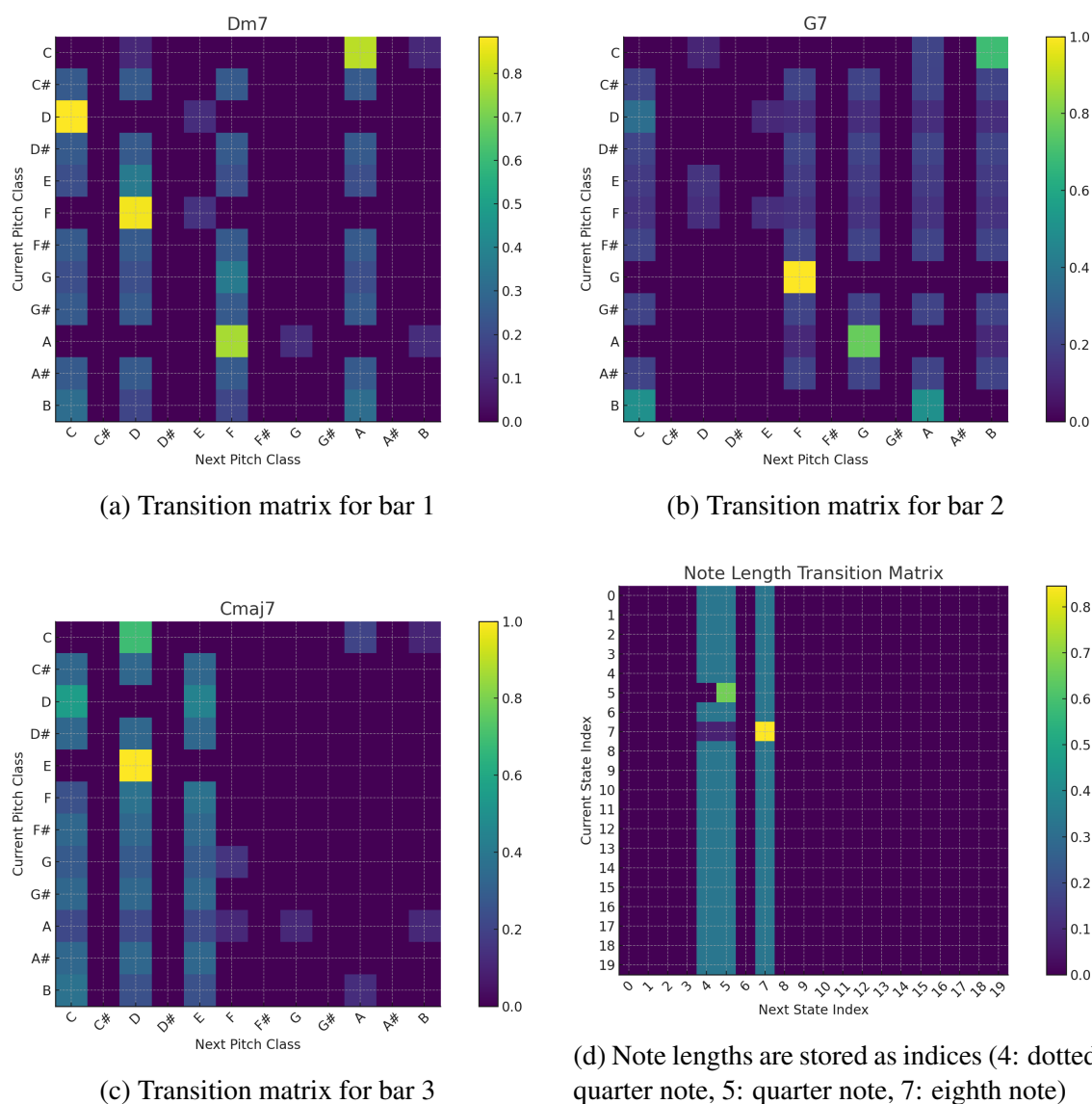


Figure 3.3: All transition matrices parsed from the original lick

3.3 Creator

The Creator class is responsible for taking the transition matrices and outputting a sequence of tuples: (pitch_class, note_length). It operates by generating one bar at a time, starting from a random chord tone and note length. By setting the final state to the start state at the end of a loop, the last note of the previous bar connects to the first note of the next bar, ensuring that the transitions between bars is smooth. For example, this is a numerical representation of the output lick. It's formatted as (pitch class, note length), where pitch classes range from 0-11 (C, C#, D, ..., B) and note length is the length of a note in ticks.

[[(2, 240), (4, 240), (5, 240), (2, 240), (0, 240), (9, 240), (5, 240), (2, 240)], [(7, 240), (5, 240), (0, 480), (11, 480), (9, 480)], [(2, 480), (4, 240), (2, 240), (0, 240), (2, 240), (0, 240), (2, 240)]]

3.4 Embellisher

The embellisher is responsible for making sure that the music is interesting. One functionality of the embellisher is turning a quarter note into a triplet motion toward the next beat if it's a chord tone. Here's an example of how it would change the notes from above:

The image displays two musical staves in 4/4 time, with a tempo of 120 bpm. The chords are Dm7, G7, and Cmaj7. The top staff shows the original lick, which consists of a sequence of quarter notes. The bottom staff shows the embellished version, where the quarter notes are replaced by triplet eighth notes, creating a more rhythmic and interesting sound.

Figure 3.4: Before and After Embellishing the Lick

4 Results

4.1 Current User Experience

The user is able to use any developer tool to create a midi file and place it into the inputs folder. The implementation requires a MIDI file (.mid) with n bars and a list of n chords. The program will not run unless the chords are valid in the musicaiz style - pitch class concatenated with a chord quality (e.g. "Cmaj7" or "Bbdominant seventh flat five").

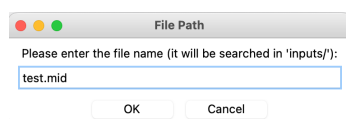


Figure 4.1: Box that searches for the midi file

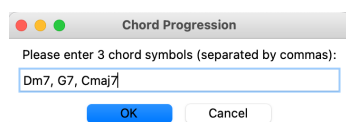


Figure 4.2: Box that tells you how many chords to input

Once the user clicks "OK," the program will execute and automatically open the new MIDI file in MuseScore on their computer.

4.2 Examples

Here are the first 4 outputs from inputting the original lick from above, which were gathered in about 2 minutes.



Figure 4.3: Original inputted lick

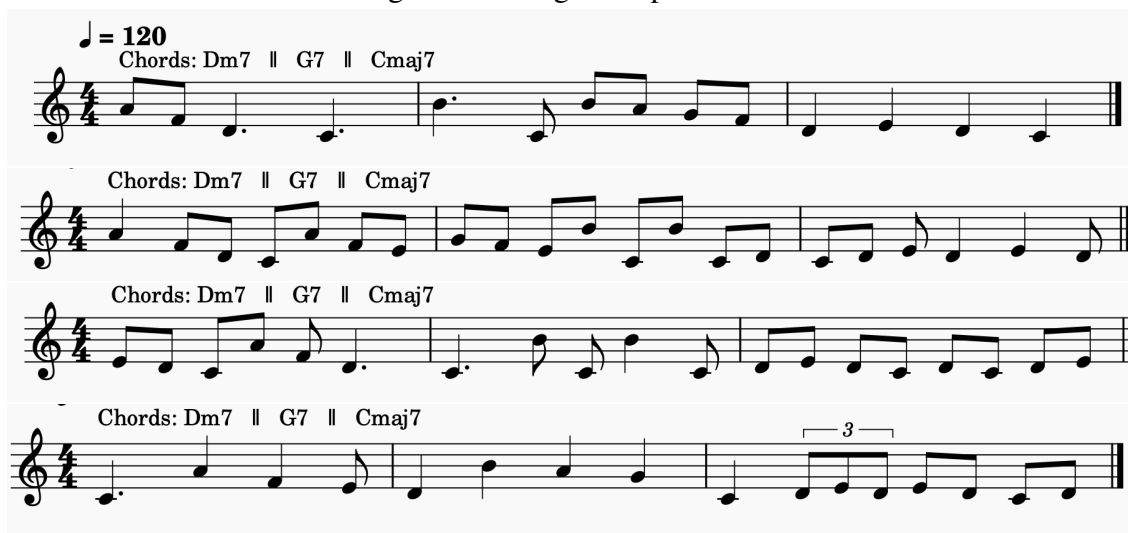


Figure 4.4: Quick example outputs, taking the original lick as an input

4.3 Discussion

It is almost impossible to quantify music - what sounds good to one person may be uninteresting to another. With this in mind, we prioritized the ability for the user to rapidly generate and compare multiple outputs of the same re-imagined lick. Because the primary aim of the project is to help an aspiring jazz musician re-evaluate the lines they play, we set more defined boundaries on the notes that are allowed to be sampled during regeneration. These constraints are based on Bebop conventions (favoring chord tones or emphasizing stepwise motion). The goal is to create a tool that can improve the way a beginner approaches and conceptualizes improvisation.

As users become more experienced, the value of the model shifts. For a beginner, it can offer insights into how the melody interacts with the harmonic structure beneath it. For an advanced player however, the model may be too predictable or limited in its ability to play outside the structure. This is not a flow - this is the intended trajectory. The purpose of the system is not to be an endless source, but a temporary tool to help a musician get out of a creative rut. Once the model has become obsolete, the user has grown as a musician and can revisit the program once they've inevitably found themselves in a rut once again.

5 Related Work

Probabilistic models for music have been in place for decades, even before computers were widely available, algorithmic composition was employed by composers such as Iannis Xenakis. In 2008, Jean-Francois Paient wrote a thesis on the feasibility of different probabilistic models for both music generation and prediction.[1]

Ilana Shapiro also used Markov chains for music generation, but in the realm of classical music. Shapiro’s work constructs a probabilistic model from symbolic input data, encodes both pitch and rhythmic information into “sound objects,” and uses transition matrices to produce music that stylistically resembles the input.[2] Due to its simplicity and flexibility, we found it a very applicable method for the realm of jazz. However, we felt the need to include an embellisher as well as add more complexity to the creation of the transition matrices in parsing and regeneration.

Other methods for music generation involve deep learning and follow a self-learning approach. Yadav et al. use a lightweight deep learning model for jazz music generation. [3] Our models are similar in that we both use MIDI to symbolically hold data. However, these models necessitate MIDI data in larger batches - for a beginner musician, it may be difficult to collect large amounts of MIDI files since most audio to MIDI converters are fairly recent, AI-powered, and not the most reliable.

6 Conclusions

This research addressed the question of whether algorithmic composition techniques could effectively support jazz musicians learning to improvise by generating stylistically coherent improvisational licks from minimal musical input. The results clearly demonstrated that the Markov model successfully produced musically coherent and altered jazz improvisations that carries aspects of the original lick provided. This underscores the efficacy of leveraging probabilistic models with limited data, proving that a simple model only using data input by a musician can perform. By offering a practical tool developed in Python, this project has the potential to contribute to jazz education, providing musicians with an accessible, interpretable method to practice and enhance their improvisational skills.

7 Future Work

There are two main limitations of the current code. 1) In choosing to use one-line MIDI data as the input, we lose the ability to read in other types of data, such as text. 2) Because there is no direct link between the chord changes and the music, the user can currently only have one chord per bar. To circumvent this issue, when having multiple chord changes in a bar, it is currently necessary for the user to elongate the notes in the lick.

For example:



Figure 7.1: The original lick

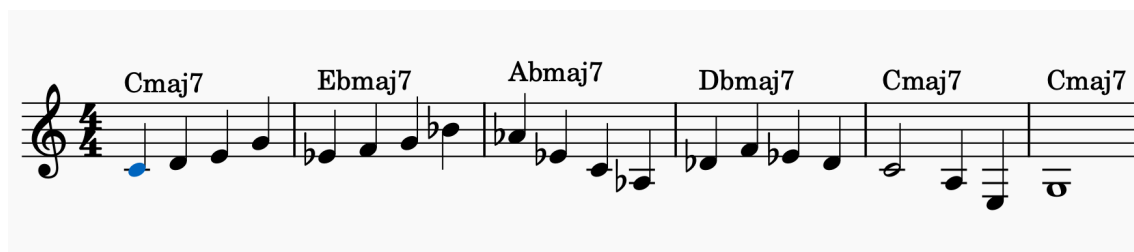


Figure 7.2: How a user must input the information, doubling the note lengths



Figure 7.3: A sample output of a regenerated lick

Future work involves creating a plugin for MIDI-supported UI (such as FL Studio or MuseScore) that can strip the data from the score and use that as inputs to the model.

Jazz also features lots of triplets - it will become important for those to be dealt with in an elegant manner. One potential maneuver is to make the Markov Chain for rhythm multiple orders — that is, $P(X = x|x_1, x_2, \dots, x_n) = P(X = x|x_n - 2, x_n - 1, x_n)$. In other words, each event is reliant on the three states before it, so that when a triplet is sampled, the triplet is kept in a group of three.

This project relies on a hybrid of formalized rule structures and informal musical intuition, sidestepping the need for data-intensive models such as Hidden Markov Models. However, the direct next steps for development of the lick modifier involve the implementation of further arbitrary rule systems — modular, interpretable frameworks that grow organically from theory-informed heuristics, stylistic idioms, and user-defined constraints.

For instance, the current model includes one instance of rule layering: Embellishments are conditionally added based on both rhythmic and harmonic context. If a quarter note lands on a downbeat and the surrounding chord tones allow, insert a triplet turn; otherwise, retain the original note. Rule layering is an effective tool to ensure "realistic" embellishing, and having more instances of this technique alongside other enhancements are the next steps to improving the lick modifier.

These enhancements could involve additional parameters for the user. For example, density parameters determine how often embellishments appear or how "dense" a lick becomes rhythmically and melodically. These systems could also include style selectors that activate different embellishment methods based on genre-appropriate conventions, such as enclosure patterns in bebop or specific rhythmic cells in Afro-Cuban styles. Each style could define rules for when and how to insert passing tones, neighbor notes, anticipations, and dissonant extensions, enabling more idiomatic variations from the minimal input. These parameters would also allow for a wider variety of expressive outputs, decided by the user, without additional input data.

By making the rule-based system more arbitrary, future works could bridge the gap between hard-coded rules and self-learning models — leveraging domain expertise without requiring large training datasets, and giving users more transparency and control over the generative process.

Bibliography

- [1] JEAN-FRANCOIS PAIEMENT. “Probabilistic Models for Music”. PhD thesis. University of Montreal, 2008.
- [2] Ilana Shapiro. “Markov Chains for Music Generation”. link to repository: <https://github.com/ilanashapiro/Model-Music-Generation>. Last commit Jan 1, 2021.
- [3] Prasant Singh Yadav et al. “A Lightweight Deep Learning-Based Approach for Jazz Music Generation in MIDI Format”. In: *Lightweight Deep Learning Models for Resource Constrained Devices* (2022).